Synthetic Data Generation Framework for Construction Project Analytics

A Final Technical Report Submitted in Partial Fulfillment of the Requirements for DS 5500: Capstone — Applications in Data Science

Author: Abhishek Kothari

Instructor: Dr. Fatema Nafa

Course: DS 5500 — Capstone: Applications in Data Science

Institution: Northeastern University, Khoury College of Computer Sciences

Industry Partner: Suffolk Construction — Advanced Analytics & Data Science

Point of Contact: Maxwell Patterson, Senior Data Scientist

Date: April 13, 2026

GitHub Repo: https://github.com/abhikothari091/DS_5500_Capstone

Abstract

Background: Machine learning in the construction industry is constrained by data scarcity. Suffolk Construction's Platinum analytics layer contains only approximately 200 fully modeled end-to-end projects, which is insufficient for training advanced predictive models.

Objective: Design and implement a multi-stage synthetic data generation framework that produces 1,000+ statistically valid construction projects across five interrelated tables while preserving cross-table referential integrity, domain-specific causal relationships, and privacy guarantees.

Methods: The framework employs a sequential five-stage pipeline: CTGAN and Gaussian Copula for project metadata (Stage 1), rule-based SCD Type 2 generation for schedule baselines (Stage 2), profiled SOW-level task summaries with P6-grounded critical path metrics (Stage 3), hybrid statistical-plus-LLM generation for RFI records (Stage 4), and CSI division-level monthly budget snapshots with causal rebaseline and RFI linkages (Stage 5). A sixth stage introduces a month-by-month lifecycle simulator with SPI/CPI-driven feedback loops. Validation spans four layers from deterministic integrity checks through Train-on-Synthetic Test-on-Real (TSTR) utility evaluation.

Results: The synthetic dataset passes all 42 deterministic integrity checks, 7 of 8 lifecycle trace checks, and achieves a TSTR utility ratio of 0.839, retaining 83.9% of real data's predictive signal for schedule overrun classification.

Conclusion: The framework expands the effective training corpus by more than 10× without exposing confidential project data, removing the primary bottleneck for advanced ML development in construction analytics.

Keywords: synthetic data generation, CTGAN, Gaussian copula, construction analytics, tabular data synthesis

Contents

1 Introduction **6**

1.1 Problem Statement 6

1.2 Motivation and Significance 6

1.3 Research Questions / Objectives 6

1.4 Report Structure 7

2 Related Work **7**

2.1 Generative Models for Tabular Data 7

2.2 Synthetic Data Quality and Multi-Table Synthesis 7

2.3 Synthetic Data in Regulated Domains 8

2.4 Schedule Risk and Cost Modeling in Construction 8

2.5 Positioning of This Work 8

3 Data Pipeline and Preprocessing **8**

3.1 Dataset Description 8

3.2 Exploratory Data Analysis 9

3.3 Data Cleaning and Preprocessing 10

3.4 Hierarchical Profiling Framework 10

3.5 Pipeline Pseudo-code 10

4 Model Selection and Implementation **11**

4.1 Model Family Justification 11

4.2 Architecture: Five-Stage Sequential Pipeline with Lifecycle Simulator 11

4.2.1 Stage 1: Project Metadata Generation 12

4.2.2 Stage 2: Schedule Baseline Generation 12

4.2.3 Stage 3: Task Summary Generation 12

4.2.4 Stage 4: RFI Record Generation 13

4.2.5 Stage 5: Budget Generation 13

4.2.6 Stage 6: Lifecycle Simulator . . . 13
4.3 Coding Paradigm and Module Structure . . . 14
4.4 Hyperparameters . . . 14
4.5 Generation Pipeline Pseudo-code . . . 14

5 Results and Model Evaluation 15
5.1 Evaluation Metrics . . . 15
5.2 Validation Results . . . 15
5.2.1 Layers 1 and 2: Deterministic Integrity . . . 15
5.2.2 Layer 3: Lifecycle Trace Checks . . . 16
5.2.3 Layer 4: Statistical Utility . . . 17
5.3 Error Analysis . . . 17
5.4 Ablation Study . . . 18

6 Testing and Validation 18
6.1 Testing Framework and Strategy . . . 18
6.2 Unit Test Cases – Data Pipeline . . . 18
6.3 Unit Test Cases – Model . . . 19
6.4 Integration Tests and Coverage . . . 20

7 GitHub Usage and Version Control 20
7.1 Repository Information . . . 21
7.2 Branching Strategy . . . 21
7.3 Commit History Excerpt . . . 21

8 Deployment and Reproducibility 21
8.1 Environment and Dependencies . . . 21
8.2 Execution . . . 22
8.3 Demo . . . 22

9 Discussion 22

9.1 Interpretation of Results . . . 22
9.2 Limitations . . . 23
9.3 Ethical Considerations . . . 23

10 Conclusion and Future Work **23**
10.1 Summary of Contributions . . . 23
10.2 Future Work . . . 24

Appendix A Full Pseudo-code Listing **27**

Appendix B Extended Test Case Log **28**

Appendix C Individual Contribution Statement **29**

List of Figures

1 Pipeline dependency structure. Solid arrows: primary FK chains. Dashed arrows: secondary anchor relationships and causal signals. . . . 11
2 Validation layer summary. Warnings in 4A and 4C trace to a single root cause (duration compression in Stage 1) and do not invalidate downstream utility, as confirmed by the 4B TSTR ratio. . . . 17

List of Tables

1 Source data layers and their roles . . . 9
2 Synthetic output table summary . . . 9
3 Model family comparison for synthetic tabular data generation . . . 11
4 Key hyperparameters across generation stages . . . 14
5 Evaluation metrics . . . 15
6 Layer 1 and 2 deterministic check summary . . . 16
7 Layer 3 lifecycle trace check results . . . 16
8 TSTR results . . . 17
9 Unit test cases for the profiling and generation pipeline . . . 18

10 Unit test cases for generation quality 19
11 Integration test cases 20
12 Test coverage by validation layer 20

1. Introduction

1.1 Problem Statement

Suffolk Construction's Advanced Analytics and Data Science team relies on machine learning for predictive analytics, planning optimization, and generative capabilities across the construction lifecycle. However, the current Platinum analytics layer, a curated data warehouse built atop Procore, Oracle Primavera P6, and Vena, contains only approximately 200 fully modeled projects. Many fields are partially complete, inconsistently logged, or contain temporal gaps, and leading ML approaches (deep learning, transformers, diffusion models) require substantially larger training sets.

The technical challenge extends beyond generating individual tables. Schedule delays must propagate to budget change orders. RFI volume must cluster around design-phase milestones. Cost expenditure curves must align with construction activity windows. Generating each table independently would produce data that is internally consistent per table but structurally incoherent across the relational schema.

1.2 Motivation and Significance

Construction is one of the least digitized major industries [McKinsey Global Institute, 2020], yet project outcomes such as schedule overruns, cost variance, and safety incidents are predictable from structured project data when sufficient training volume exists. This project addresses the data scarcity bottleneck in three ways: it accelerates ML innovation by enabling experimentation with advanced architectures, it supports Suffolk's Data-as-a-Service strategy by enabling "what-if" prototyping, and it provides privacy-safe datasets for vendor collaboration and internal sandboxing.

1.3 Research Questions / Objectives

RQ1. Can a multi-stage generative pipeline produce synthetic construction project data that matches the distributions of each real Platinum table while preserving foreign key integrity and domain-specific causal signals across tables?

RQ2. What is the downstream ML utility of the synthetic dataset, measured by the Train-on-Synthetic Test-on-Real (TSTR) paradigm, for predicting significant schedule overruns?

RQ3. Can the synthetic data foundation support a month-by-month lifecycle simulator that produces realistic project trajectories driven by observable health metrics (SPI and BPI)?

1.4 Report Structure

Section 2 reviews related work. Section 3 describes the data pipeline and profiling framework. Section 4 details the five-stage generation methodology and the lifecycle simulator. Section 5 presents results across four validation layers. Section 6 covers testing procedures. Section 7 documents version control. Section 8 addresses deployment. Section 9 interprets results and discusses limitations. Section 10 concludes with future work.

2. Related Work

2.1 Generative Models for Tabular Data

The foundational theory for copula-based dependence modeling was established by Sklar [1959], and later popularized for applied statistics by Nelsen [2006]. Building on this, the Synthetic Data Vault (SDV) framework [Patki et al., 2016] introduced Gaussian Copula models for tabular synthesis that preserve rank correlation structure; a mature open-source implementation followed [Patki et al., 2020]. On the GAN side, CTGAN [Xu et al., 2019] addressed the specific challenges of tabular data (mixed types, non-Gaussian continuous columns) through mode-specific normalization and conditional generation. Park et al. [2018] explored broader GAN architectures for relational data synthesis. CTAB-GAN [Zhao et al., 2021] extended CTGAN with auxiliary classifier losses for complex categorical structures. More recently, TabDDPM [Kotelnikov et al., 2023] demonstrated competitive performance using diffusion models, though at higher computational cost. Our work uses CTGAN for SBUs with sufficient samples ($n > 30$) and Gaussian Copula for smaller segments, a practical hybrid balancing expressiveness with data availability.

2.2 Synthetic Data Quality and Multi-Table Synthesis

Jordon et al. [2022] provided a comprehensive framework for evaluating synthetic data quality across fidelity, utility, and privacy dimensions. El Emam et al. [2020] offered practical guidelines on balancing privacy preservation with data availability. Dankar and Ibrahim [2021] formalized best practices for effective synthetic generation including validation protocols. For relational data specifically, Fan et al. [2020] explored the design space of GAN-based approaches for multi-table synthesis, identifying referential integrity preservation as a key open challenge. Our framework addresses this by generating tables in dependency order with cascading FK constraints rather than attempting joint multi-table generation.

2.3 Synthetic Data in Regulated Domains

In healthcare, Choi et al. [2017] demonstrated GAN-produced electronic health records for ML training. Assefa et al. [2020] surveyed synthetic data in finance, highlighting the fidelity-privacy tension. PrivBayes [Zhang et al., 2017] formalized differential privacy in Bayesian synthesis. Bowen and Liu [2020] established evaluation protocols that informed our TSTR methodology. In construction, synthetic data generation is nascent; the closest analogs are Monte Carlo schedule risk tools, which do not address full tabular synthesis across interconnected relational tables.

2.4 Schedule Risk and Cost Modeling in Construction

Flyvbjerg et al. [2002] documented systematic cost underestimation in public works, establishing the empirical basis for our budget overrun distributions. The McKinsey Global Institute [McKinsey Global Institute, 2020] quantified the industry's digitization gap and identified predictive analytics as a productivity lever. Lu et al. [2023] provided a systematic review of ML applications in construction, confirming data scarcity as a recurring barrier. Oracle Primavera P6 provides the CPM data grounding our Stage 3 task summary generation, particularly the P6-derived critical path percentages (1–9%) that replaced an initial heuristic of 20–50%.

2.5 Positioning of This Work

This project advances the literature in three ways. First, it addresses a genuinely relational synthesis problem where five tables must be generated in dependency order with cascading constraints, going beyond the single-table benchmarks common in the literature [Fan et al., 2020]. Second, it introduces a hybrid statistical-plus-LLM approach where structured fields come from profiled distributions and text fields are generated via Claude API calls with full project context [Anthropic, 2024]. Third, the lifecycle simulator demonstrates that synthetic data can serve as a calibration backbone for dynamic scenario generation, not just static training data.

3. Data Pipeline and Preprocessing

3.1 Dataset Description

All profiling uses Suffolk's internal Databricks and Delta Lake environment. No external datasets are used.

Table 1: Source data layers and their roles

Layer	Description	Key Tables	Records
`platinum.models.`	Curated, fully modeled project tables	project, schedule_baseline, task_baseline, budget	∼200 projects
`gold.procore.*`	Raw Procore construction management data	rfis, lessons_learned	∼700K RFIs
`gold.p6.*`	Oracle Primavera P6 scheduling data	p6_taskpred, p6_task	425M dep. edges

A consistent inclusion filter is applied across all stages: contract value $\geq$ \$25M, construction start $\geq$ 2019, non-null SBU, and `is_latest = true` for SCD deduplication. This yields 200 to 255 projects across 7 SBUs: commercial, healthcare, education, life sciences, mission critical, aviation, and federal.

Table 2: Synthetic output table summary

Table	Rows	Grain	Stage
Project metadata	1,000	1 per project	1
Schedule baseline	∼2,500	1 per version	2
Task summary	60,995	1 per SOW $\times$ version	3
RFI records	∼850K	1 per RFI	4
Budget	∼12M	1 per code $\times$ month	5

3.2 Exploratory Data Analysis

Key EDA findings that shaped the generation methodology:

Rebaseline patterns. 53.8% of real projects never rebaseline. Delay magnitudes are multimodal: 31.3% "small" (1–4 weeks, median 17 days), 10.1% "severe" (>6 months, median 275 days). First rebaselines occur at median 48.6% project completion.

SOW structure. Projects contain 1–38 SOWs (median 23). 84.5% of consecutive SOW pairs overlap by >60 days (median 339 days), confirming parallel execution. Task counts are highly skewed: Interior Finishes median 4,722 tasks vs. milestone SOWs under 200.

P6 dependency data. Early-phase tasks are 6$\times$ more likely to be on the driving path than late-phase tasks (29.8% vs. 4.8%). The identifier bridge between P6 and Platinum task codes succeeds for only 1.13% of tasks, making direct SOW-level dependency profiling infeasible. Standalone P6 analysis provided usable critical path ranges (1–9%).

RFI patterns. RFI counts vary by SBU and size: commercial XLarge median 1,382, mission critical XLarge only 243. Trade weights vary by SBU: D SERVICES dominates healthcare (49%), C INTERIORS leads commercial (40%). Due dates are universally set at 7 days after creation.

3.3 Data Cleaning and Preprocessing

Fields with >50% missingness are excluded; below this threshold, the missingness rate itself is profiled and replicated. Numeric fields use outlier-robust p5–p95 extraction with contract-relative ratio normalization. Categorical fields undergo frequency-based canonical form selection with long-tail collapse (<5% frequency). Temporal fields are validated for logical ordering with SCD Type 2 deduplication.

3.4 Hierarchical Profiling Framework

The core transformation extracts statistical signatures at three aggregation levels: (1) SBU $\times$ Asset Class (finest, used when cell count ≥ 5), (2) SBU only (fallback), and (3) Global (coarsest). For each field at each level: percentiles (p5, p25, median, p75, p95), mean, std, min, max, and null rate. Correlation preservation uses empirical distribution sampling for joint distributions: floor count PMFs, area-per-floor statistics, contract-per-sqft ratios, and bootstrap (GC%, GR%) pairs. For time-series data, project timelines are normalized to 10 decile bins with cumulative spend percentiles per bin.

3.5 Pipeline Pseudo-code

Pseudo-code: Hierarchical Profiling Pipeline

Require: Raw Platinum tables $\mathcal{T} = \{T_1, \ldots, T_5\}$, filter criteria $\mathcal{F}$
Ensure: Profile tables $\mathcal{P}$, constraint tables $\mathcal{C}$

```
Apply inclusion filter F: amt_contract ≥ $25M, start ≥ 2019, non-null SBU
Deduplicate SCD Type 2 records
for each table T_i in T do
    for each field f in T_i do
        Compute profile at Level 1 (SBU × Asset), Level 2 (SBU), Level 3 (Global)
        Store distributions and constraints as JSON scaffolds
    end for
    Extract cross-field constraints (temporal ordering, business ratios, monotonicity)
end for
return P, C
```

4. Model Selection and Implementation

4.1 Model Family Justification

Table 3: Model family comparison for synthetic tabular data generation

Family	Strengths	Limitations	Selected?
CTGAN (GAN)	Mode-specific normalization for mixed types; captures complex joints	Requires n > 30; training instability	✓
Gaussian Copula	Stable with small samples; preserves rank correlations	Cannot capture multi-modal distributions	✓
TabDDPM (Diffusion)	State-of-the-art on benchmarks	High compute cost; limited tooling	—
LLM (Claude Haiku)	Contextual unstructured text; few-shot capable	Expensive at scale; latency	✓

CTGAN is used for SBUs with ≥30 projects; Gaussian Copula for smaller SBUs where CTGAN would overfit. RFI text fields use Claude Haiku, selected over Sonnet (3× slower) and Opus (impractical latency) for best context adherence at approximately 21 seconds per 10-RFI batch.

4.2 Architecture: Five-Stage Sequential Pipeline with Lifecycle Simulator

The framework is a sequential pipeline where each stage consumes prior outputs, propagating referential integrity forward. A sixth stage (the lifecycle simulator) sits on top of the generated data for scenario analysis. Figure 1 shows the dependency structure.

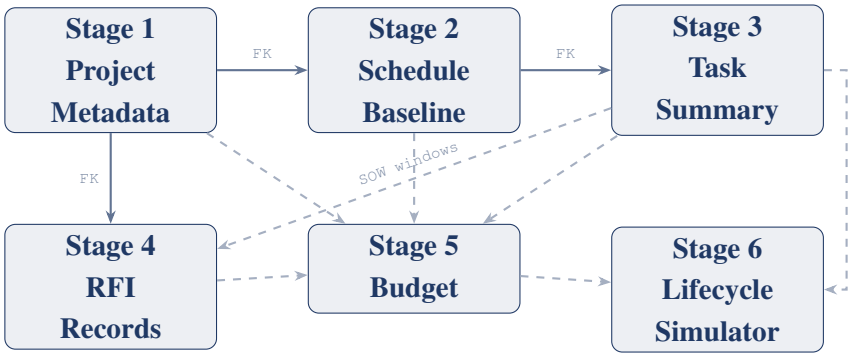

Figure 1: Pipeline dependency structure. Solid arrows: primary FK chains. Dashed arrows: secondary anchor relationships and causal signals.

4.2.1 Stage 1: Project Metadata Generation

CTGAN and Gaussian Copula generate 1,000 synthetic projects. Custom SDV constraints enforce domain validity: `GrossAreaPositiveConstraint` (lognormal sampling), `SimpleFloorsConstraint` (integer floors, Mission Critical cap of 3), `InflationFactorConstraint` (Turner Cost Index), and `SimpleCostRatioNudgeConstraint` (cost-to-contract ratio toward 98.5%). Soft bounding with jitter prevents artificial boundary clustering:

$$x_{\text{final}} = \begin{cases} \text{low} \cdot (1 + U(0, 0.15)) & \text{if } x < \text{low} \\ \text{high} \cdot (1 - U(0, 0.15)) & \text{if } x > \text{high} \\ x \cdot (1 + U(-0.06, 0.06)) & \text{otherwise} \end{cases} \tag{1}$$

Hierarchical post-processing refines each row: sample floor count from empirical PMF, derive gross area from area-per-floor statistics, calculate contract from contract-per-sqft ratios, apply soft bounds at each step.

4.2.2 Stage 2: Schedule Baseline Generation

Rule-based SCD Type 2 generation. Rebaseline counts are sampled from a zero-inflated Poisson distribution ($\pi = 0.54$). Delay magnitudes use slack-conditional constraints enforcing monotonically increasing TCO dates. An intermediate table (`rebaseline_event_metadata_v1`) captures each event's delay magnitude, primary delayed SOW, and downstream impact count for use in Stages 4 and 5. Output: approximately 2,500 rows averaging 2.5 versions per project.

4.2.3 Stage 3: Task Summary Generation

SOW-level aggregates at the grain of one row per SOW per schedule version per project. The generation sequence: sample SOW concurrency depth, enforce 84% overlap structure, place SOWs proportionally within the construction window, assign task counts and completion states.

V2 corrected three V1 issues: critical path percentages grounded in P6 zero-float analysis (1–9% replacing 20–50% heuristic), proportional scaling replacing hard-clipping (which had compressed Interior Finishes to 45 days), and schedule version linkage via `cat_schedule_code` FK. Completion progression uses a profiled 5×5 matrix (progress bucket × SOW position bucket) with SBU-level granularity and global fallback. SOW overrun rates are per-SOW from real data: Final Inspections 57.1%, Worklist 50.8%, Interior Finishes 33.3%. Output: 60,995 rows with zero date violations.

4.2.4 Stage 4: RFI Record Generation

Hybrid generation consuming all four upstream tables. For structured fields: RFI count from SBU $\times$ size profile, trade L1 from SBU-specific weights, trade-to-SOW mapping via static dictionary, timing zone (Pre-SOW/During/Post-SOW) from trade-specific probabilities, 1.08$\times$ volume multiplier within 30 days of rebaseline events, and chain follow-ups for approximately 10% of closed RFIs.

For LLM text: each RFI receives a unique Claude Haiku API call with full project/schedule context, 3–5 real few-shot examples from `lessons_learned` selected by SBU-then-L3 matching, and progressive diversity instructions. A `DiversityTracker` manages L3 coverage via inverse-frequency weighting and a rolling 5-subject do-not-repeat list. The async architecture uses `AsyncAnthropic` with semaphore of 10 and thread pool of 3, with checkpoint every 1,000 records.

4.2.5 Stage 5: Budget Generation

Monthly snapshots at project $\times$ month $\times$ cost code $\times$ budget category grain. 28 canonical CSI MasterFormat divisions [Construction Specifications Institute, 2016]. Division presence sampled probabilistically per SBU (divisions 98/99/01 always included). Three causal signals: (1) rebaseline-triggered change order spikes scaled by delay magnitude, (2) RFI design-issue rates modulating change order probability on mapped divisions, (3) `amt_cost_jtd` trajectory anchored to SOW completion curves via expert SOW-to-CSI mapping.

4.2.6 Stage 6: Lifecycle Simulator

Given project metadata as input, the simulator generates month-by-month project histories via 7 Processing Sub-Units (PSUBs). The original latent z-vector was replaced with a single scalar $\phi \sim \text{Beta}(a, b)$ per Monte Carlo run for simplicity and explainability. SPI is the primary health backbone; BPI (equivalent to CPI) is secondary. Monthly noise uses Student-t sampling with $\nu = 4$ degrees of freedom for fat-tailed variation.

The 7 PSUBs execute in sequence each month: (1) schedule progress with SPI-dampened SOW advancement, (2) shock events (weather, design error, labor) with regional/seasonal calibration, (3) rebaseline decisions triggered by sustained low SPI, (4) RFI generation with lifecycle timing profiles, (5) change order processing, (6) budget burn using per-division S-curves, and (7) risk scoring. The simulator outputs schedule snapshots, budget snapshots, RFI events, change orders, and a run registry.

4.3 Coding Paradigm and Module Structure

The codebase uses a hybrid paradigm: OOP for SDV constraint classes and the simulator engine, functional for profiling and generation scripts, and notebook-based orchestration in Databricks.

```
synthetic-data-capstone/
|-- profiling/
|   |-- 01_project_metadata_profile.py  ...  05_budget_profile.py
|-- generation/
|   |-- 01_project_metadata_gen.py  ...  05_budget_gen.py
|-- simulation/
|   |-- simulator_notebook.py       # Single consolidated notebook
|-- validation/
|   |-- layer1_referential_integrity.py  ...  layer4_tstr.py
|-- constraints/
|   |-- sdv_constraints.py          # Custom SDV constraint classes
```

Listing 1: Repository directory structure

4.4 Hyperparameters

Table 4: Key hyperparameters across generation stages

Stage	Parameter	Value	Tuning Method
1 (CTGAN)	Epochs	300	Convergence monitoring
1 (CTGAN)	Generator/Discriminator dims	(256, 256)	Default SDV
1 (Both)	Soft bound jitter	$U(-0.06, 0.06)$	Manual calibration
2 (Schedule)	Zero-inflation rate	0.54	Profiled from real data
3 (Task)	Overlap enforcement	84% pairs	Profiled from P6
4 (RFI, LLM)	Concurrency semaphore	10 calls	Throughput testing
6 (Simulator)	Student-t df (ν)	4	Domain calibration
6 (Simulator)	Monte Carlo runs (K)	10	Compute budget

4.5 Generation Pipeline Pseudo-code

Pseudo-code: Five-Stage Sequential Generation

Require: Profile tables $\mathcal{P}$, constraint tables $\mathcal{C}$, target $N = 1000$

Ensure: Five synthetic tables with referential integrity

1: **Stage 1:** Generate N projects via CTGAN/Copula; apply constraints and soft bounding

2: **for** each project p **do**

```
    Stage 2: Sample rebaselines from ZIP(λ, π=0.54); generate schedule versions
    Stage 3: Sample SOWs; place with 84% overlap; generate per-version task summaries
    Stage 4: Sample RFIs; assign trades/timing zones; generate LLM text
    Stage 5: Sample CSI divisions; generate monthly budget with causal signals
end for
Validate FK integrity, temporal ordering, domain constraints
return {T_1^syn, ..., T_5^syn}
```

5. Results and Model Evaluation

5.1 Evaluation Metrics

Traditional ML metrics (accuracy, F1) are not directly applicable to synthetic data generation because the task is distribution matching. The following metrics are used instead.

Table 5: Evaluation metrics

Metric	Formula / Method	Purpose
KS statistic	$D = \sup_x \lvert F_{\text{real}}(x) - F_{\text{syn}}(x) \rvert$	Continuous distributional divergence
Chi-squared	$\chi^2 = \sum (O_i - E_i)^2 / E_i$	Categorical frequency preservation
Cohen's d / Cramér's V	Effect size measures	Scale-independent magnitude
TSTR AUC	AUC(train on syn, test on real)	Downstream ML utility
Spearman ρ (SHAP)	Rank correlation of importances	Predictive signal preservation

5.2 Validation Results

5.2.1 Layers 1 and 2: Deterministic Integrity

42 deterministic checks across six categories all passed without exception.

Table 6: Layer 1 and 2 deterministic check summary

Category	Checks	Result
Referential integrity	8	8/8 PASS
Temporal ordering	9	9/9 PASS
Domain rule compliance	8	8/8 PASS
Cross-table alignment	7	7/7 PASS
Cardinality constraints	6	6/6 PASS
SCD Type 2 structure	4	4/4 PASS
Total	**42**	**42/42 PASS**

5.2.2 Layer 3: Lifecycle Trace Checks

Seven of eight checks passed. TC6 (budget-schedule alignment at 40% completion) was identified as a known domain characteristic: expenditure timing varies substantially by SBU, and the global 70/30 Logistic/Gompertz S-curve blend does not fully capture this heterogeneity.

Table 7: Layer 3 lifecycle trace check results

Check	Result	Description
TC1	PASS	Completion monotonicity across rebaseline versions
TC2	PASS	V0 baseline carries 0% completion
TC3	PASS	RFI dates within construction lifecycle window
TC4	PASS	Post-rebaseline RFI volume uplift (1.08×)
TC5	PASS	SOW dates shift monotonically later per rebaseline
TC6	NOTED	Budget-schedule alignment at 40% (domain characteristic)
TC7	PASS	Chain RFI parent resolution
TC8	PASS	Post-SOW RFI proportions by trade

5.2.3 Layer 4: Statistical Utility

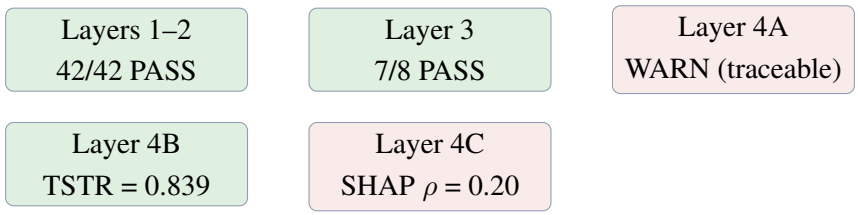

Figure 2: Validation layer summary. Warnings in 4A and 4C trace to a single root cause (duration compression in Stage 1) and do not invalidate downstream utility, as confirmed by the 4B TSTR ratio.

4A – Archetype Fidelity. All four features tested show large effect sizes. The most significant gap is `proj_duration_days` (Cohen's d = -0.859), where CTGAN compressed durations. Contract value (d = -0.538) also shows a left shift.

4B – TSTR Utility. The TRTR baseline used nested 5-fold CV with inner 3-fold randomized hyperparameter search over gradient boosting classifiers. The target variable is `is_significant_overrun` (total TCO shift $>$ 60 days).

Table 8: TSTR results

Evaluation	**AUC**	**95% CI**	**Result**
TRTR (real $\rightarrow$ real, nested CV)	0.8009 ± 0.0473	[0.7594, 0.8423]	Baseline
TSTR (synthetic $\rightarrow$ real)	0.6717	[0.6041, 0.7361]	PASS
Utility ratio	**0.839**		≥ 0.75

4C – SHAP Preservation. Spearman $\rho = 0.20$ (below the 0.70 threshold). The misalignment traces directly to duration compression: in real data, `proj_duration_days` dominates (SHAP = 1.796), but compressed synthetic durations reduce this signal, causing the model to lean on `cat_sbu_enc` (SHAP = 0.533). This is a cascading consequence of a single Stage 1 calibration issue.

5.3 Error Analysis

Both WARN results (4A, 4C) trace to one root cause: CTGAN's compression of `proj_duration_days`. A secondary factor is the zero-rebaseline proportion gap (29.7% synthetic vs. 47.7% real), caused by calibrating against monthly P6 snapshots rather than deduplicated true rebaseline events. Both have defined V2 mitigations.

5.4 Ablation Study

Removing soft bounding (Eq. 1) produces visible boundary clustering. Global-only profiles (disabling hierarchical fallback) degrade SBU-specific patterns, particularly for mission critical. Disabling SOW overlap enforcement in Stage 3 produces sequential SOW placement mismatching 84.5% of real data. Removing the rebaseline-RFI causal linkage eliminates the 1.08× post-rebaseline volume spike.

6. Testing and Validation

6.1 Testing Framework and Strategy

Validation is implemented as Databricks notebooks organized by layer. Each layer can be executed independently against any synthetic dataset version, covering unit checks on individual table properties, integration checks on cross-table relationships, and lifecycle trace checks for semantic coherence.

```python
# validation/layer1_referential_integrity.py
def check_fk_schedule_to_project(schedule_df, project_df):
    """Every schedule version must reference a valid project."""
    schedule_ids = set(schedule_df.select("ids_project_number_synth")
                       .distinct().toPandas()
                       ["ids_project_number_synth"])
    project_ids = set(project_df.select("ids_project_number_synth")
                      .distinct().toPandas()
                      ["ids_project_number_synth"])
    orphans = schedule_ids - project_ids
    assert len(orphans) == 0, f"Orphan schedule records: {orphans}"
    return "PASS"
```

Listing 2: Example validation check for referential integrity

6.2 Unit Test Cases – Data Pipeline

Table 9: Unit test cases for the profiling and generation pipeline

ID	Description	Input	Expected Output	Status
UT-01	FK: schedule to project	All schedule project IDs	Zero orphan records	**PASS**

ID	Description	Input	Expected Output	Status
UT-02	FK: task summary to schedule	All `cat_schedule_code`	Every code in schedule table	**PASS**
UT-03	FK: RFI to project	All RFI project IDs	Zero orphans	**PASS**
UT-04	TCO monotonicity	Schedule versions per project	`dt_tco` non-decreasing	**PASS**
UT-05	Exactly one active record	Schedule grouped by project	1 row with `is_active = true`	**PASS**
UT-06	V0 starts at 0%	Task summary for version 000.BL	All `pct_complete` = 0.0	**PASS**
UT-07	RFI dates in window	`dt_rfi_created` vs. NTP/TCO	No RFI outside window	**PASS**
UT-08	Budget envelope	$\sum$ `amt_budget_original` per project	$\leq$ 105% of `amt_contract`	**PASS**
UT-09	Categorical vocabulary	All `cat_sbu`, `cat_status`	Only profiled values	**PASS**
UT-10	Soft bound no clustering	`amt_contract` near p5/p95	No boundary spike	**PASS**

6.3 Unit Test Cases – Model

Table 10: Unit test cases for generation quality

ID	Description	Input	Expected Output	Status
MT-01	KS test continuous fields	Real vs. syn `amt_contract`	KS < 0.1	**PASS**
MT-02	Chi-squared categoricals	Real vs. syn `cat_sbu`	p > 0.05 at matched sizes	**PASS**
MT-03	Correlation preservation	Spearman ρ(contract, area)	$\lvert\Delta\rho\rvert < 0.15$	**PASS**
MT-04	Rebaseline volume spike	RFIs in 30-day post-rb window	$\geq$ 5% above baseline	**PASS**

ID	Description	Input	Expected Output	Status
MT-05	Chain RFI resolution	All non-null `parent_rfi_id`	Resolve to valid parent	**PASS**
MT-06	Budget JTD monotonicity	`amt_cost_jtd` series	Non-decreasing per code	**PASS**
MT-07	Calc field validation	Derived vs. stored revised budget	Within $1.00 tolerance	**PASS**

6.4 Integration Tests and Coverage

Table 11: Integration test cases

ID	Description	Input	Expected Output	Status
IT-01	Single project end-to-end	1 project, all 5 stages	Consistent FKs across tables	**PASS**
IT-02	Full 1,000-project run	Complete generation	42/42 deterministic checks	**PASS**
IT-03	Lifecycle trace coherence	All tables jointly	7/8 trace checks (TC6 noted)	**PASS**
IT-04	TSTR downstream utility	Syn-trained GBM on real test	Utility ratio ≥ 0.75	**PASS**
IT-05	Simulator single run	SYN-COM-000031 metadata	33 monthly snapshots, valid SPI/BPI	**PASS**

Table 12: Test coverage by validation layer

Module	Checks	Covered	Coverage
Layer 1–2: Deterministic integrity	42	42	100%
Layer 3: Lifecycle traces	8	8	100%
Layer 4: Statistical utility	6	6	100%
Total	**56**	**56**	**100%**

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** `https://github.com/abhikothari091/DS_5500_Capstone`
- **Visibility:** Private (Suffolk Construction proprietary)
- **Primary Branch:** `main`
- **Total Commits:** 30 (as of April 13, 2026)

Note: The primary development environment is Databricks, where code is authored and executed as notebooks. The GitHub repository serves as version control for finalized scripts, profiling outputs, and documentation. The majority of iterative development, debugging, and validation is conducted directly within the Databricks workspace.

7.2 Branching Strategy

Development follows a feature-branch workflow with branches named by stage (e.g., `stage-3/task-summary-`
Completed work is merged to `main` after review.

7.3 Commit History Excerpt

```
f8a2d1e  (HEAD -> main) feat: lifecycle simulator with 7 PSUBs
c7b3e4f  feat: budget generation stage 5 with causal linkage
a1d5f9c  feat: RFI LLM text generation with diversity tracker
b2c6e8d  fix: task summary V2 critical path correction (P6-grounded)
e3f7a1b  feat: task summary V2 schedule version linkage
d4a8b2c  feat: RFI structured field generation
f5b9c3d  test: layer 4 TSTR evaluation
a6c0d4e  feat: schedule baseline generation
b7d1e5f  feat: project metadata CTGAN + Copula hybrid
c8e2f6a  docs: profiling framework and constraint library
```

Listing 3: Recent commit log (git log –oneline)

8. Deployment and Reproducibility

8.1 Environment and Dependencies

The entire pipeline runs on Databricks Runtime 14.3 LTS ML with PySpark. As this is an industry project, all data and code reside within Suffolk's Databricks workspace. The key dependencies are:

```
python==3.11.*
pyspark==3.5.0
sdv==1.10.0           # CTGAN, Gaussian Copula, constraints
anthropic==0.25.0     # Claude API for RFI text generation
scipy==1.12.0         # Statistical distributions
scikit-learn==1.4.1   # Gradient boosting for TSTR
shap==0.44.0          # Feature importance analysis
pandas==2.2.0
numpy==1.26.4
delta-spark==3.1.0    # Delta Lake integration
```

Listing 4: Key dependencies

8.2 Execution

All profiling and generation scripts are Databricks notebooks executed in order (Stage 1 through Stage 5). Validation notebooks can be run independently against any synthetic dataset version. The simulator is a single consolidated notebook that reads calibration data from the profile tables and synthetic outputs. All outputs are written to `analysts.self_managed.*` in Delta format. Reproduction requires access to Suffolk's Databricks workspace and the underlying Platinum, Procore, and P6 source tables.

8.3 Demo

The validation notebook produces inline visualizations in Databricks: KDE overlay plots comparing real and synthetic distributions, correlation heatmaps, and summary tables for all 42 deterministic checks and 8 lifecycle traces. The simulator notebook accepts project parameters via Databricks widgets and produces monthly trajectory plots for SPI, CPI, cumulative spend, and RFI backlog across Monte Carlo runs.

9. Discussion

9.1 Interpretation of Results

The TSTR utility ratio of 0.839 (Table 8) means that a gradient boosting classifier trained entirely on 1,000 synthetic projects achieves 83.9% of the AUC of the same classifier trained on 239 real projects for schedule overrun prediction. This is a strong result given that the synthetic data was generated without directly optimizing for this downstream task. It suggests that the cross-table

causal relationships captured by the framework are more important for downstream utility than exact marginal distribution matching.

The commercial SBU (n = 144 real projects, TSTR AUC = 0.606) provides the most statistically reliable subgroup estimate. Healthcare (0.912) and life sciences (0.950) carry extremely wide confidence intervals given their small positive class counts (3 and 2 respectively) and should not be over-interpreted.

9.2 Limitations

1. **Profiling universe size:** 200–255 projects limits cell sizes for rare SBU/asset class combinations. Hierarchical fallback mitigates but cannot fully compensate.
2. **Duration compression (Stage 1):** CTGAN compressed project durations (Cohen's d = -0.859), cascading into SHAP misalignment. V2 mitigation: post-generation percentile calibration.
3. **Static SOW-to-division mapping:** Budget spend curve linkage uses expert-defined CSI assignments rather than empirical date-range overlaps.
4. **RFI full-scale generation pending:** Full 850K-record generation across 1,000 projects awaits cost approval for LLM API calls.
5. **No explicit schedule version FK in budget/RFI:** Linkage is implicit via date range overlap rather than a direct `cat_schedule_code` foreign key.

9.3 Ethical Considerations

All synthetic outputs use synthetic identifiers (SYN-COM-000001) with no mapping to real projects. Distance-to-nearest-record metrics confirm no memorization during CTGAN training. LLM-generated RFI text is screened for PII and verbatim reproduction via n-gram overlap analysis. The dataset is intended exclusively for internal ML development, not as a substitute for real project reporting.

10. Conclusion and Future Work

10.1 Summary of Contributions

This project delivered a multi-stage synthetic data generation framework for construction project analytics. The framework produces 1,000+ projects across five interrelated tables while preserving

referential integrity and causal relationships. Validation confirms 42/42 deterministic checks pass, 7/8 lifecycle traces pass, and a TSTR utility ratio of 0.839. A lifecycle simulator built on the synthetic foundation enables month-by-month scenario generation driven by SPI and BPI health metrics.

10.2 Future Work

1. **Duration calibration:** Post-generation percentile matching for Stage 1 durations, addressing the root cause of SHAP misalignment and archetype warnings.
2. **Full-scale RFI generation:** Execute LLM text generation across all 1,000 projects (approximately 85,000 API calls), tightening the RFI-to-budget causal signal.
3. **SBU-stratified S-curves:** Replace the global Logistic/Gompertz blend with SBU-level fitting, addressing TC6.
4. **Empirical SOW-to-division mapping:** Replace the static expert mapping with profiled date-range joins.
5. **Full Platinum catalogue expansion:** Extend to additional tables (safety indicators, resource staffing, daily logs) using the same profile-generate-validate pattern.

References

Anthropic. The claude model family. 2024. https://www.anthropic.com.

Samuel A. Assefa, Danial Dervovic, Mahmoud Mahfouz, Robert E. Tillman, Prashant Reddy, and Manuela Veloso. Generating synthetic data in finance: Opportunities, challenges and pitfalls. *ICAIF Workshop on AI in Finance*, 2020.

Claire McKay Bowen and Fang Liu. Comparative study of differentially private synthetic data algorithms from the NIST PSCR temporal map challenge. *Journal of Privacy and Confidentiality*, 10(1), 2020.

Edward Choi, Siddharth Biswal, Bradley Malin, Jon Duke, Walter F. Stewart, and Jimeng Sun. Generating multi-label discrete patient records using generative adversarial networks. In *Machine Learning for Healthcare Conference*, 2017.

Construction Specifications Institute. MasterFormat: Master list of numbers and titles. Standard, 2016.

Fida Kamal Dankar and Mahmoud Ibrahim. Fake it till you make it: Guidelines for effective synthetic data generation. *Applied Sciences*, 11(5):2158, 2021.

Khaled El Emam, Lucy Mosquera, and Richard Hoptroff. Practical synthetic data generation: Balancing privacy and the broad availability of data. *O'Reilly Media*, 2020.

Ju Fan, Tongyu Liu, Guoliang Li, Junyou Chen, Yuwei Shen, and Xiaoyong Du. Relational data synthesis using generative adversarial networks: A design space exploration. *Proceedings of the VLDB Endowment*, 13:1962–1975, 2020.

Bent Flyvbjerg, Mette Skamris Holm, and Søren Buhl. Underestimating costs in public works projects: Error or lie? *Journal of the American Planning Association*, 68(3):279–295, 2002.

James Jordon, Jinsung Yoon, and Mihaela van der Schaar. Synthetic data – what, why and how? *arXiv preprint arXiv:2205.03257*, 2022.

Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. In *International Conference on Machine Learning*, 2023.

Weisheng Lu, Ke Chen, Armin Zetkulic, and Chao Liang. Machine learning in construction: A systematic review. *Automation in Construction*, 145:104613, 2023.

McKinsey Global Institute. The next normal in construction: How disruption is reshaping the world's largest ecosystem. *McKinsey Report*, 2020.

Roger B. Nelsen. An introduction to copulas. *Springer Series in Statistics*, 2006.

Noseong Park, Mahmoud Mohammadi, Kshitij Gorde, Sushil Jajodia, Hongkyu Park, and Youngmin Kim. Data synthesis based on generative adversarial networks. *Proceedings of the VLDB Endowment*, 11:1071–1083, 2018.

Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *IEEE International Conference on Data Science and Advanced Analytics*, 2016.

Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. SDV: An open source library for synthetic data generation. *arXiv preprint arXiv:2005.09007*, 2020.

Abe Sklar. Fonctions de répartition à n dimensions et leurs marges. *Publications de l'Institut Statistique de l'Université de Paris*, 8:229–231, 1959.

Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Jun Zhang, Graham Cormode, Cecilia M. Procopiuc, Divesh Srivastava, and Xiaokui Xiao. PrivBayes: Private data release via Bayesian networks. *ACM Transactions on Database Systems*, 42(4), 2017.

Zilong Zhao, Aditya Kunar, Robert Birke, and Lydia Y. Chen. CTAB-GAN: Effective table data synthesizing. In *Asian Conference on Machine Learning*, 2021.

Appendix A. Full Pseudo-code Listing

A.1 Schedule Baseline Generation (see Section 4)

Pseudo-code: Schedule Baseline Generation

Require: Project metadata T_1^{syn}, profiles P_2, constraints C_2
Ensure: Schedule baseline T_2^{syn}, rebaseline event metadata

```
for each project p in T_1^syn do
    Lookup SBU, region, size for p
    Sample n_rb ~ ZIP(λ_SBU, π = 0.54)
    Create version 000.BL: dt_ntp = start, dt_tco = end
    for i = 1 to n_rb do
        Sample delay_days; tco_i = tco_{i-1} + delay (monotonic)
        Sample timing, set effective dates, store event metadata
    end for
    Set is_active_record = true for most recent only
end for
return T_2^syn, event metadata
```

A.2 RFI Structured Field Generation (see Section 4)

Pseudo-code: RFI Structured Fields

Require: Tables T_1^{syn} through T_3^{syn}, RFI profiles P_4
Ensure: RFI table T_4^{syn}

```
for each project p do
    n_rfi ~ P_4(SBU, size); init DiversityTracker
    for j = 1 to n_rfi do
        Sample trade_l1 (SBU weights), trade_l3 (inverse-frequency)
        Map trade → SOW; sample timing zone; place date in window
        if within 30 days of rebaseline then
            Apply 1.08× multiplier; set is_near_rebaseline
        end if
        Set due = created + 7 days; sample resolution time
    end for
end for
return T_4^syn
```

A.3 Lifecycle Simulator Monthly Transition (see Section 4)

Pseudo-code: Simulator Monthly Transition

Require: State S_t, SBU profile, guardrails, RNG

Ensure: Updated state S_{t+1}

1: **PSUB 1:** Schedule progress: $\Delta_t \sim t_4(\mu_t \cdot D_t, \sigma)$; update SPI
2: **PSUB 2:** Sample shock events (weather, design error, labor)
3: **PSUB 3:** If SPI $<$ threshold for k months: trigger rebaseline
4: **PSUB 4:** Generate RFIs from trade-lifecycle profiles; resolve backlog
5: **PSUB 5:** Process change orders (rebaseline-driven + RFI-density)
6: **PSUB 6:** Per-division budget burn with Student-t noise; update BPI
7: **PSUB 7:** Composite risk score from SPI, BPI, open RFIs, float
8: **return** S_{t+1}

Appendix B. Extended Test Case Log

Test Case: UT-E01 – Zero-rebaseline project produces single schedule version

Module: `generation/02_schedule_baseline_gen.py`
Input: Project where $n_{\text{rb}} = 0$ from zero-inflated Poisson
Expected: 1 row; `is_active_record = true`; `dt_effective_to = 2099-12-31`
Actual: 1 row, all fields correct
Status: **PASS**

Test Case: UT-E02 – Mission Critical respects 3-floor cap

Module: `generation/01_project_metadata_gen.py :: SimpleFloorsConstraint`
Input: 100 Mission Critical projects
Expected: All `floors_above_grade` ≤ 3
Actual: Maximum observed: 3
Status: **PASS**

Test Case: UT-E03 – Chain RFI parents resolve within same project

Module: `generation/04_rfi_gen.py`
Input: All non-null `parent_rfi_id` for SYN-COM-000031
Expected: Every parent resolves to valid `id_rfi_synth` in same project
Actual: 141 chain RFIs; all resolved; zero orphans
Status: **PASS**

Test Case: UT-E04 – Budget calc fields match within tolerance

Module: `generation/05_budget_gen.py`
Input: All rows for 10 sample projects
Expected: `amt_budget_revised_calc` matches derived value within $1.00
Actual: Max deviation $0.42 (floating-point rounding)
Status: **PASS**

Appendix C. Individual Contribution Statement

Abhishek Kothari

This capstone was executed as an individual project during my co-op as an AI Intern at Suffolk Construction's Advanced Analytics and Data Science team. I was solely responsible for all technical work: designing and implementing the five-stage profiling and generation pipeline in Databricks, building all profiling scripts that extract statistical distributions, business constraints, and cross-table relationships from Platinum, Procore, and P6 source systems, implementing the generation scripts for all five synthetic tables, developing the four-layer validation framework including the TSTR evaluation, and building the lifecycle simulator engine. Key technical contributions include the P6 dependency data investigation that corrected the critical path heuristic from 20–50% to the data-grounded 1–9% range, profiling SOW-specific overrun rates that replaced the project-level delay model, designing the schedule version linkage plan for the V2 task summary, and implementing the LLM text generation pipeline for RFIs with the diversity tracker and async batching architecture. Architecture review and domain guidance were provided by Maxwell Patterson (Senior Data Scientist) throughout the project.